

CSE 5106: Multi-agent Soccer Simulation Project

AUSTIN BEINDER*, JC STINSON, and ALEXANDER WANG^{✉*}, Washington University in St. Louis, USA

Multi-agent soccer simulation is a well-studied testbed for cooperative and competitive multi-agent systems. We propose a custom physics-based soccer environment implemented in PyBullet for studying multi-agent reinforcement learning. Development will proceed incrementally, beginning with simplified tasks such as keep-away and single-agent on single-agent scenarios before progressing to 2v2 or higher agent-count matches. We plan to investigate centralized-training/decentralized-execution methods and compare learned agents against fixed-policy opponents. The final system should provide a controllable environment for evaluating coordination, role specialization, and emergent team behavior.

ACM Reference Format:

Austin Beinder, JC Stinson, and Alexander Wang. 2026. CSE 5106: Multi-agent Soccer Simulation Project. 1, 1 (September 2026), 3 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

We aim to create a continuous, physics-based multi-agent soccer simulation. We plan to utilize PyBullet, which is a 3D, physics-based simulation environment. In this 3D environment we will lock the objects into a single plane to behave as if it were 2D. PyBullet has been selected as the base simulation environment because it is capable of being run in a "headless" configuration (i.e. only simulation interactions) which will accelerate a reinforcement learning training process, but also can produce a video graphics interface for model performance evaluation. Our game will employ a multitude of rules inspired by real soccer. The goal would be to get teams of agents to work together for objective-oriented solutions against an opposing team of agents.

A "game"'s outcome is to be defined by parameters of a reward function and performance capabilities of the "player" agents. The players will be split between two teams: red and blue. Blue will be MARL-trained agents, and red will be a fixed policy agent (for stationarity of reinforcement learning latent space). This fixed policy will either be pre-scripted (i.e. a simple proportional controller forcing the agent towards the ball, and then towards the goal if in possession of the ball) or a previous trained model of blue agents. The players are then trained via RL to "play" the game per the defined parameters.

The project would likely be in a finished state when we have a game of 2v2 or 3v3 soccer with successful, converged training, measurable improvement over initial baseline performance and successful coordination between agents. Should this be achieved earlier in the semester than expected we will pursue things like making the game take place in a fully 3D six degree-of-freedom space like the Ender's Game battle arena, or adding more players and specialized roles to learn such as goalies.

*All authors contributed equally to this research.

Authors' Contact Information: Austin Beinder, a.beinder@wustl.edu; JC Stinson, j.c.stinson@wustl.edu; Alexander Wang, w.yuanji@wustl.edu, Washington University in St. Louis, St. Louis, Missouri, USA.

2 Discussion of related work

Several related papers with similar ideas have been found in the literature. The RoboCup paper by Qin et al. for instance utilizes MARL in a 2D environment to model a team dynamic [3]. This paper builds upon an existing RoboCup 2D simulation environment and establishes MARL in a client-server environment while using Python for its learning algorithms. Learning algorithms it compares include MAPPO, QMIX, and ParaDQN. We would likely want to use its recommendations of learning algorithms as a source of inspiration, as well as its architecture or recommended action spaces. In contrast to Qin et al.'s work we will be implementing our own custom simulation environment in PyBullet and focusing more on lower agent count scenarios.

Another related paper by Naji et al. [2] spends a lot of time comparing multiple role types or team configurations for their own simulation "STMAS". An example of different role types they define is "defense agents" vs. "attack agents", each with subcategories of specialists such as a goalkeeper, or a midfielder, etc. This might be useful should we work our way up to a 3v3 MAS where one agent is a goalie. It could also be useful to reference if the agents need to swap training modes when in an offensive state vs. a defensive state.

Additionally, as noted during the second lecture there was a paper by Christopher Amato with in-depth introduction to a range of MARL methods [1]. We anticipate using this paper as a guide when looking at Centralized Training for Decentralized Execution (CTDE) algorithms. For instance, it has dedicated sections on MAPPO and QMIX, which are both discussed in Qin et al., but Amato also discusses many other CTDE algorithms to choose from.

3 Solution Approach

The means of building the simulator environment will begin with the core physics simulation in PyBullet with a "gym" intermediate layer inspired by the likes of OpenAI's RL gym platform and similar environments to handle interfacing with the MARL process. The method of the MARL process will be informed by what seems most suitable to the low agent count continuous space as informed by the likes of Qin et al. and Amato's work and tuned by hand or using "stock" parameters initially.

The definition of the "game" and its rules will be handled primarily by the reward function for the MARL process, as well as input from state handling in the gym layer and parameters of the player agents and game object ball. Here we discuss an example of how we foresee each player, or the ball, will publish its state such that the agents know where everyone is, and we list some of the possible actions or interactions that might occur.

3.1 Game Entities

Each game entity such as the player agents or the game ball will need to have certain parameters kept track of individually, and also have parameters visible as state information to other agents. For example, each player will publish its unique id, team affiliation, velocity in x and y, and position in x and y, as well as if they are controlling the ball. Each player will have the capability of "kicking" the ball if it is within a certain control radius. We anticipate this kick to be the ability to impart a chosen velocity in terms of heading and magnitude upon the ball. Players will also need to keep track of if they collide with another team's player while in control of the ball. If this happens, control of the ball will have a 50% chance of flipping to the other team. We anticipate each player also having possible action to accelerate in a chosen vector and magnitude, though we might change this movement regime to velocity directly or augment it with added friction if acceleration makes them resemble hockey pucks too much.

3.2 Rewards and Training

For training we are thinking we may utilize IPPO or MAPPO but are still investigating possibilities. We will want to use an RL method that is suited to continuous state, continuous action space such as soft actor critic (SAC) proximal policy optimization (PPO), deep deterministic policy gradient (DDPG), or Twin Delayed DDPG (TD3). High level search indicates there are centralized training/decentralized execution versions of each, and will use existing research to select what appears to be optimal for our use case.

While training, we will seek to reward desired behaviors and penalize undesired behaviors. For instance we will reward the agent for kicking the ball towards the opposing goal, and reward it a lot for kicking it into the opposing goal. To encourage cooperation we could add a reward term for if the agent passes the ball to a teammate when contextually appropriate such as distance from opposing team players or also reward if the ball doesn't change possession between teams often.

If necessary, we could further encourage "optimal" behavior by penalizing the inverse of what we reward to make reward signaling of desirable game states or play even clearer. Examples would be scrimmage and collision with opposing team agents, losing positive control of the ball, and proximity of the ball to own team goal.

4 Anticipated Results

Projected build up of results and demonstration:

- Phase 0: Get all components working together. Single blue agent against fixed policy red agent in environment to validate environment, gym wrapper, and first RL algorithm training loop. Simple "game". Perhaps "keep away".
- Phase 1: Two blue agents with centralized training/decentralized execution (CTDE) to validate MARL loop - possibly add more red fixed agents to increase complexity of the problem being solved.
- Phase 2: 3v3 (or n v n) with different roles (offense, defense), maybe contextual role switching. Test scalability of MARL method to higher player count or more complex game states.
- Phase 3: Scalability with first algorithm of more agents/more complex game states (more agents, more balls, more goals, 3D Ender's Game arena, etc.)
- Endgame: Examining/comparing other algorithms against fixed set of game rules.
- Anticipated Outcome: We anticipate that an interesting demo would be possible after all of these phases. We would be hopeful of at least achieving Phase "0" by the time of the midterm report. Phase 2 would be considered an the minimum acceptable place to end the project. If we were to end on Phase 2, it would be possible to write a report showcasing an interesting demo, and/or comparing the outcome of different algorithms.

References

- [1] Christopher Amato. 2025. An Initial Introduction to Cooperative Multi-Agent Reinforcement Learning. arXiv:2405.06161 [cs.LG] <https://arxiv.org/abs/2405.06161>
- [2] Areen Naji, Amjad Rattout, and Rashid Jayousi. 2023. Using a Soccer team as a test bed for multi-agent systems simulation. *Proceedings of the International Conference on Advanced Technologies* 11 (Sep. 2023), 109–115. doi:10.58190/icat.2023.26
- [3] Haobin Qin, Baofeng Zhang, Hidehisa Akiyama, and Keisuke Fujii. 2026. R2D-RL: A RoboCup 2D Soccer Environment for Multi-Agent Reinforcement Learning. arXiv:2606.18786 [cs.AI] <https://arxiv.org/abs/2606.18786>

Received 12 September 2026; revised 12 September 2026; accepted 12 September 2026